

VM/370

Single Processor Mode

Installation and Use Guide

Version 1.0.0

12 June 2025

Contents

Introduction to VM/370 Single Processor Mode	3
Requirements.....	4
Installation Summary.....	5
Installation Detailed Step-by-Step.....	6
SPMODE Commands	13
Setting up the MVS Virtual Machine	14
Understanding Channel Sets.....	15
Bringing up MVS in SPMODE.....	17
Comparison of MVS Performance with and without SPMODE.....	19
Discussion of Timing Results	19
Limitations When Using SPMODE	21
Optional Modifications for MVS 3.8 Guests	22
Loading the Enhancements from Tape	22
Installing the Enhancements.....	24
VM/370 SPMODE Version History.....	29
Other Modifications and Software.....	30

Introduction to VM/370 Single Processor Mode

Single processor mode in VM/370 (SPMODE) enables VM to confine itself to a single processor in a two-CPU configuration. This allows one CPU to be used by VM itself and one CPU for an MVS guest. In the SPMODE environment, VM/370 runs as a uniprocessor system (that is, not an attached processor (AP) system). The SPMODE implementation allows CP to use real and virtual prefixing in a way that permits an MVS guest to run natively on the second processor and without any of the overhead of CP on that processor. But because VM/370 is in control of the complex, MVS can still do VM-desirable things such as spooling JES2 output to CMS users, using the virtual reader, or dialing and dropping terminals for TSO use.

In a VM/370 SPMODE environment, MVS actually runs on both processors. On processor 0, MVS runs under the control of CP in a virtual machine and uses whatever CPU resources that CP gives it, and on processor 1 MVS runs with full control of that processor. Therefore, in this environment, MVS can actually use more than one CPU's worth of processing power.

By allowing MVS to run natively on the second processor, the considerable overhead of VM is avoided. On the second processor, MVS is free to issue privileged instructions as needed without CP simulation, and i/o and paging operations can be performed directly by MVS without CP's translations and simulation overhead. Therefore, there is an incentive to try to move as many of MVS's i/o devices to CPU 1 as possible. It is very simple to do that, and this guide will show you how. It is possible to have MVS run at better than 90% of its native speed, while still running under the control of VM/370.

MVS 3.8 is the only operating system that we have available that can run in SPMODE under VM/370. Later versions of MVS had the capability, but VM/370 does not have the necessary support to run later versions of MVS, specifically the 370E features. MVS 3.8J can already run in SPMODE as distributed in builds such as TK4- and TK5, as well as other builds if all of the available MVS PTFs are applied.

SPMODE was introduced in later versions of VM in the 1980s. I was never able to use the feature back in the day, because we only had a single CPU on a 4831. Instead, we ran MVS/SP under VM/SP using the Preferred Machine Assist feature. But since then, I've always been intrigued by the ideas around SPMODE and the way it works and later hoped to be able to get it to work in VM/370 under Hercules someday.

Bob Polmanter
June 2025

Requirements

The following are the requirements that must be met in order to install and operate SPMODE:

- A working VM/370 Release 6 operating system. This guide is geared towards the VM/CE releases (1.2 specifically), but other distributions such as VM Sixpack or VM 5-pack will also work. The installation is tailored to VM/CE 1.2 but the process for the other distributions is almost identical.
- VM/370 Shadow Table Bypass Support is required as a prerequisite to SPMODE. This is modification number HRC068DK. Most VM/370 users already have this support as it was preinstalled on distributions of VM Sixpack 1.3 and all VM/CE releases.
- A working MVS 3.8 operating system that can be run as a guest of VM/370. The TK5 distribution is recommended, but other distributions or privately generated systems will work if they are current with all available PTFs.

Installation Summary

Below is a summary of the installation steps required:

1. Load the SPMODE materials onto MAINT's disks.
2. Update the DMKHRC MACLIB.
3. Assemble the affected support modules.
4. Set the V=R area size.
5. Build the CP nucleus.
6. Re-IPL CMS and grab the loader map.
7. Shutdown VM/370 and Hercules. Modify the Hercules configuration to use two processors.
8. Bring up Hercules and VM/370 and test SPMODE.

Users experienced in building a CP nucleus should be able to complete all of these steps in about 15 minutes. Someone unexperienced should be able to complete the steps in about 30 minutes by carefully following all of the instructions. It's very simple if you just follow the process.

Installation Detailed Step-by-Step

The **spmodev100.aws** tape contains 1 file, in VMFPLC2 format. If you wish to inspect the contents of the tape files prior to loading them onto your maintenance minidisks, a 1-cylinder t-disk will be more than sufficient to hold the files.

The installation instructions are oriented towards the VM/CE 1.x systems.

As always, please make sure you have a good backup of your VM DASD before applying any of this modification.

Prerequisites:

The SPMODE materials are modification HRC081DK. Modification HRC068DK (Shadow Table Bypass Support) is required as a prerequisite. HRC068DK is already preinstalled on Sixpack 1.3 and all VM/CE releases.

Tape File 1:

Contains the CP modifications for the SPMODE support. Included are all of the HRC081DK update files for this support and their associated AUXHRC files. The tape file also contains a small EXEC file to assist with the installation.

SPECIFIC INSTALLATION STEPS

It is recommended you make a backup copy of your Hercules VM disks prior to installing this modification. If you are using shadow files, you only need back up the shadow files themselves.

1. Logon to the MAINT userid. Attach a tape drive (usually 480) to MAINT as virtual 181. Access the CP disks and then access the 094 disk as A and load the contents of the first file of the tape using VMFPLC2.

```
logon maint
attach 480 maint 181
(make the spmodev100.aws tape ready to Hercules)
vmsetup cp
acc 094 a
vmfplc2 load * * a
```

The 094 disk should remain as A through the completion of step 5.

Note: If you are not using VM/CE and do not have the VMSETUP EXEC, access CP's disks in this way:

```
acc 094 a
acc 194 c/c
acc 294 d/d
acc 394 e/e
```

An example of the expected results for step 1 are shown below:

L MAINT

ENTER PASSWORD:

DASD 19D LINKED R/W; R/O BY 002 USERS
DASD 19E LINKED R/W; R/O BY 004 USERS
DASD 190 LINKED R/W; R/O BY 004 USERS
FILES: NO RDR, 002 PRT, 001 PUN
LOGON AT 20:45:33 GMT THURSDAY 03/20/25
VM Community Edition V1 R1.2

Y (19E) R/O
U (19D) R/O
B (5E5) R/O
Ready; T=0.01/0.01 20:44:30
att 480 * 181
TAPE 480 ATTACH TO MAINT 181
Ready; T=0.01/0.01 20:44:37

vmsetup cp
E (594) R/O
F (094) R/O
G (194) R/O
H (294) R/O
I (394) R/O
Ready; T=0.01/0.03 20:44:41

ac 094 a
'094' REPLACES 'A (191) '
'094 F/A' RELEASED
Ready; T=0.01/0.01 20:50:37

vmfplc2 load * * a
LOADING.....
ASM81 EXEC A1
DMKBLD AUXHRC A1
DMKBLD HRC081DK A1
DMKCFC AUXHRC A1
DMKCFC HRC081DK A1
DMKCFO AUXHRC A1
DMKCFO HRC081DK A1
DMKCFP AUXHRC A1
DMKCFP HRC081DK A1
DMKCFS AUXHRC A1
DMKCFS HRC081DK A1
DMKCPR AUXHRC A1
DMKCPR HRC081DK A1
DMKCPV AUXHRC A1
DMKCPV HRC081DK A1
DMKCQY AUXHRC A1

```
DMKCQY HRC081DK A1
DMKDMP AUXHRC A1
DMKDMP HRC081DK A1
DMKDSP AUXHRC A1
DMKDSP HRC081DK A1
DMKEMA AUXHRC A1
DMKEMA HRC081DK A1
DMKPRV AUXHRC A1
DMKPRV HRC081DK A1
DMKPSA AUXHRC A1
DMKPSA HRC081DK A1
DMKUSO AUXHRC A1
DMKUSO HRC081DK A1
PSA AUXHRC A1
PSA HRC081DK A1
VMBLOK AUXHRC A1
VMBLOK HRC081DK A1
END-OF-FILE OR END-OF-TAPE
Ready; T=0.01/0.05 20:53:10
```

2. Update the DMKHRC MACLIB. Issue the VMFMAC command to allow the maclib update process to proceed. You will have to page through a number of lines of update output. The name DMKHRC is specified twice on the VMFMAC command as shown.

```
vmfmac dmkhrc dmkhrc
```

Example output (partially snipped here for size):

```
vmfmac dmkhrc dmkhrc
UPDATING 'SYSID MACRO A1'.
APPLYING 'SYSID HRC108DK A1'.
SYSID MACRO ADDED.
UPDATING 'ALLOC COPY I1'.
APPLYING 'ALLOC HRC106DK A1'.
ALLOC COPY ADDED.
UPDATING 'MICBLOK COPY I1'.
APPLYING 'MICBLOK HRC068DK A1'.
MICBLOK COPY ADDED.
UPDATING 'VMBLOK COPY I1'.
APPLYING 'VMBLOK R09952DK H1'.
APPLYING 'VMBLOK R10666DK H1'.
APPLYING 'VMBLOK R11241DK H1'.
APPLYING 'VMBLOK R13304DK H1'.
APPLYING 'VMBLOK R13658DK H1'.
APPLYING 'VMBLOK R13026DK H1'.
APPLYING 'VMBLOK R13025DK H1'.
APPLYING 'VMBLOK HRC001DK A1'.
```



```
APPLYING 'VMBLOK HRC013DK A1'.  
<..snipped for size..>  
UPDATING '$EUA MACRO A1'.  
APPLYING '$EUA HRC101DK A1'.  
$EUA MACRO ADDED.  
UPDATING '$ADDRESS MACRO A1'.  
APPLYING '$ADDRESS HRC101DK A1'.  
$ADDRESS MACRO ADDED.  
DMKHRC COPY ADDED.  
Ready; T=0.49/1.64 21:02:15
```

3. Assemble the updates. Since there are 14 modules to assemble, an EXEC is provided to do all of the assemblies. The EXEC is called ASM81. Simply type:

```
asm81
```

on the CMS command line. Each of the assemblies will be performed in turn until all are completed.

Example output (partially snipped here for size):

```
asm81  
EXEC VMFASM DMKBLD DMKHRC  
UPDATING 'DMKBLD ASSEMBLE I1'.  
APPLYING 'DMKBLD R09296DK H1'.  
APPLYING 'DMKBLD R08845DK H1'.  
APPLYING 'DMKBLD R09985DK H1'.  
APPLYING 'DMKBLD R09438DK H1'.  
APPLYING 'DMKBLD HRC101DK A1'.  
APPLYING 'DMKBLD HRC065DK A1'.  
APPLYING 'DMKBLD HRC066DK A1'.  
APPLYING 'DMKBLD HRC073DK A1'.  
APPLYING 'DMKBLD HRC081DK A1'.  
ASMBLING DMKBLD  
  
ASSEMBLER (XF) DONE  
NO STATEMENTS FLAGGED IN THIS ASSEMBLY  
File 'DMKBLD TEXT A1' not found.  
DMKBLD TXTHRC CREATED  
EXEC VMFASM DMKCFC DMKHRC  
UPDATING 'DMKCFC ASSEMBLE I1'.  
APPLYING 'DMKCFC R08928DK H1'.  
<..snipped for size..>  
APPLYING 'DMKUSO HRC013DK A1'.  
APPLYING 'DMKUSO HRC029DK A1'.  
APPLYING 'DMKUSO HRC065DK A1'.  
APPLYING 'DMKUSO HRC106DK A1'.
```

```
APPLYING 'DMKUSO HRC081DK A1'.  
ASMBLING DMKUSO
```

```
ASSEMBLER (XF) DONE  
NO STATEMENTS FLAGGED IN THIS ASSEMBLY  
File 'DMKUSO TEXT A1' not found.  
DMKUSO TXTHRC CREATED  
Ready; T=3.12/7.31 21:05:59
```

4. The installation of the SPMODE materials does not itself require a VIRTUAL=REAL area but if you wish to later run MVS under VM while using SPMODE then a V=R area is required. An 8 MB V=R area is recommended. However, while you can set the area to almost any size between 32K and about 15 MB, where an MVS guest is concerned something between 4 MB and 12 MB should be used. The more size you give to the V=R area, the better for MVS, but the cost could be higher paging rates for VM. 8 MB is a good all-around value. To set this value, use the VRSIZE command as shown below (your actions are in bold lower case):

```
vrsize  
VIRTUAL=REAL OPTION REQUIRED (YES,NO):  
yes  
STORAGE SIZE OF VIRT=REAL <MINIMUM IS 32K>:  
8192k  
08192K STORAGE SIZE FOR VIRTUAL=REAL  
IS THE ABOVE ENTRY CORRECT (YES,NO):  
yes  
Ready; T=0.01/0.01 21:06:28
```

Once the V=R area size has been set by the VRSIZE command, you need not repeat this process each time you build a CP nucleus unless you wish to change the V=R size.

5. Build the CP nucleus. Since building a nucleus with a V=R area is required in order to use SPMODE, you must use the VRLOAD load list instead of the more common CPLOAD load list. Issue the commands below.

```
purge rdr  
spool PUN *  
spool PRT *  
vmfload vrload dmkhrc  
ipl 00c clear
```

Example output for step 5:

```
purge rdr
NO FILES PURGED
Ready; T=0.01/0.01 21:09:56
spool pun *
Ready; T=0.01/0.01 21:10:01
spool prt *
Ready; T=0.01/0.01 21:10:04
vmfload vrlload dmkhrc
SYSTEM LOAD DECK COMPLETE
PUN FILE 1326 TO MAINT COPY 01 NOHOLD
Ready; T=0.08/0.35 21:10:19
ipl 00c clear
Nucleus loaded on VM50-1 --- Starting CYL=0530, last CYL used=0548
CP ENTERED; DISABLED WAIT PSW '00020000 00000012'
```

6. After the "nucleus loaded" message, re-IPL CMS and fetch the load map:

```
ipl cms
close rdr
acc 094 a
readcard cpnuc map
```

Example output for step 6:

```
ipl cms
PRT FILE 1328 TO MAINT COPY 01 NOHOLD
VM Community Edition V1 R1.2

Y (19E) R/O
U (19D) R/O
B (5E5) R/O
Ready; T=0.01/0.01 21:12:10
close rdr
Ready; T=0.01/0.01 21:12:18
acc 094 a
'094' REPLACES ' A (191) '
Ready; T=0.01/0.01 21:12:25
readcard cpnuc map
RECORD LENGTH IS '132' BYTES.
Ready; T=0.17/0.50 21:12:29
```

7. Log off MAINT, and shutdown the system, and exit Hercules. While the system is down, edit the Hercules configuration file for VM/CE (currently named vm370ce.conf as distributed) and add the following line after the MAINSIZE command:

```
MAXCPU 2
```

And edit the NUMCPU line immediately afterwards from 1 to 2. When finished, the two lines should appear as:

```
MAXCPU 2  
NUMCPU 2
```

8. Restart Hercules and bring up VM/CE. That concludes the installation of SPMODE. Please consult the sections below in this document for details on how to effectively use SPMODE.

SPMODE Commands

The SPMODE function is controlled by two CP commands: SPMODE and QUERY.

The SPMODE command format is:

SPMODE ON | OFF

ON enables the SPMODE functionality in the system.

OFF disables the functionality.

The SPMODE command is a privilege class A command. In order to enable SPMODE, *all* of the following conditions must be met:

- CP must be generated as a UP-system (uniprocessor mode). The system cannot be AP generated.
- A VIRTUAL=REAL (V=R) area must be defined.
- The V=R area must not be unlocked (via previous use of the UNLOCK V=R command).
- The V=R area can be in use by a VM userid, but no SPMODE function will be realized until MVS is re-IPLed in the V=R area after SPMODE is enabled.

In order to disable SPMODE, *one* of the following conditions must be met:

- The V=R user must logged off, or,
- The V=R user must not be in EC mode (CP SET ECMODE OFF), or,
- The code running in the V=R area is not using virtual prefixing.

In order to query the status of SPMODE, use the class G command QUERY SPMODE:

QUERY SPMODE
Q

The command will respond with:

SPMODE is active | inactive

Setting up the MVS Virtual Machine

IPLing MVS while in SPMODE is the same as IPLing MVS in any typical virtual machine. Because SPMODE requires that MVS is in the V=R area, the CP directory entry for the virtual machine must specify the VIRT=REAL option. Below is an example directory entry for MVS intended to be used under SPMODE.

```
USER MVS MVS 8M 8M ABCDEFG
OPTION ECMODE BMX REALTIMER VIRT=REAL
CONSOLE 01F 3215
SPOOL 00C 2540 READER *
SPOOL 00D 2540 PUNCH A
SPOOL 00E 1403 A
SPOOL 00F 1403 A
SPOOL 30E 1403 A
* Dedicated 2703 lines for NJE :
  DEDICATE 090 090
  DEDICATE 091 091
* Dedicated Tape Devices:
  DEDICATE 480 480
* Dedicated DASD:
DEDICATE 198 198
* 3270 devices:
  SPECIAL 0C0 3270
  SPECIAL 0C1 3270
  SPECIAL 0C2 3270
  SPECIAL 0C3 3270
  SPECIAL 0C4 3270
  SPECIAL 0C5 3270
  SPECIAL 0C6 3270
  SPECIAL 0C7 3270
```

Please note the dedicated DASD in bold above. The virtual address 198 above is for the TK5 system residence volume, TK5RES. Normally, TK5 is IPLed from address 390. But no 3390 devices are defined in VM/CE's DMKRIO at 390-39F. Instead, 3390s are defined at 190-19F so we will use one of those. If you are using TK3 or TK4-, the *DEDICATE 198 198* statement should be coded as *DEDICATE 148 148*, representing volume MVSRES.

You may be wondering where the rest of the DASD are specified. Many DASD volumes appear to be missing. This is because those other volumes will be assigned to CPU 1, and therefore will not be owned or known by CP, and therefore not defined to CP. When MVS initializes during IPL and when SPMODE is active, CP will allow MVS to activate CPU 1. During that initialization MVS will detect all of the i/o devices that belong to that processor. If SPMODE were not active, the IPL would fail because not all of the necessary DASD devices are defined to MVS in the CP directory.

The MVS IPL volume (the system residence volume) must always be defined to CP. Otherwise CP will not be able to IPL MVS in a virtual machine.

Understanding Channel Sets

In the System/370 architecture, each CPU can have its own set of channels. For example, CPU 0 could have a set of channels numbered 0-5, and CPU 1 could have its own set of channels numbered 0-5. The channels are owned by the CPU and one CPU cannot do i/o via another CPU's channel. Each CPU's group of channels is called a *channel set*. Channel set 0 belongs to CPU 0. Channel set 1 belongs to CPU 1.

In the Hercules emulator, it is possible to define i/o devices to channels that belong to a different CPU other than the default of CPU 0. Typically, a DASD device in Hercules might be defined this way:

```
149 3350 dasd/work01.149
```

By default, this DASD device will be defined to channel set 0, and therefore belong to CPU 0. The default if a channel set number is not coded is channel set 0. To code a channel set, specify the channel set number followed by a colon (:) character and then the device address as shown below:

```
0:149 3350 dasd/work01.149
```

This is functionally identical to the previous definition.

For best performance of MVS under VM while running in SPMODE, the DASD devices should be defined to channel set 1. In this way, MVS can access and service i/o operations to these devices at its native speed and bypassing all VM overhead. Therefore, to fully set up MVS in this way, define all of the DASD devices on channel set 1 (except for the IPL system residence volume, which must always be on channel set 0). For a TK5 guest system under VM, the Hercules definitions could look like these:

```
0:198 3390 dasd/tk5res.390      # MVS IPL volume must be on channel set 0
1:391 3390 dasd/tk5cat.391
1:392 3390 dasd/tk5dlb.392
1:248 3350 dasd/page00.248
1:249 3350 dasd/spool0.249
1:298 3390 dasd/tk5001.298
1:299 3390 dasd/tk5002.299
1:29A 3390 dasd/work01.290
1:291 3390 dasd/work02.291
1:292 3390 dasd/work03.292
1:293 3390 dasd/work04.293
1:190 3390 dasd/tso001.190
1:191 3390 dasd/tso002.191
1:192 3390 dasd/tso003.192
```

Note that MVS requires unit addresses to be unique. This means that you could not have, for example, a 391 unit address on channel set 0 and a 391 unit address on channel set 1. Hercules will allow it, but MVS won't.

For TK5, the IPL address is normally 390. But there are no channel 3 devices of type 3390 defined to CP in DMKRIO. One option is to add 3390s on channel 3 to DMKRIO at addresses 390-39F and build

another CP nucleus. Or even easier, simply change the address of the IPL volume to one that is already defined in DMKRIO. DASD 190-19F are already defined as 3390s in DMKRIO. In the example above we make the IPL volume 198 instead of 190 because unit address 190 is already in use by DASD volume TSO001. MVS doesn't care what the unit address of the system residence volume is, as long as it is one that is defined to MVS and that it is the correct device type. 198 fits perfectly.

Other devices besides DASD devices can also be placed on channel set 1 if you care to. You could define tape drives on channel set 1:

```
1:480 3420 *
```

You can define unit record devices similarly:

```
1:00E 1403 /path/prt00e.txt crlf
```

However, please note that if you define tapes or unit record devices to channel set 1 then VM will not be able to control them. This means that you would lose the ability to ATTACH tape drives or to have MVS issue CP SPOOL 00E TO USERID commands. If you want those abilities (often the reason that you run MVS as a guest of VM), then leave most other devices on channel set 0.

Remember, any devices that you do decide to assign to channel set 1 must not be defined in the MVS directory entry. If for example, you were to place the 00E printer on channel set 1, then remove the SPOOL 00E directory statement above.

Bringing up MVS in SPMODE

When you are ready to IPL MVS, you should first enable SPMODE by issuing the CP SPMODE ON command from the system operator console, or from a Class A userid such as MAINT. Then logon to the MVS virtual machine and issue the CP IPL command for your MVS system residence volume.

During the IPL process there are no new messages or anything identifiable that indicates MVS is running with two CPUs. However, after IPL is completed you can check whether MVS is running that way by using the DISPLAY M=CPU command. For example:

```
d m=cpu
11.01.43          IEE070I 11.01.43 CPU/CHAN STATUS 195
01 0123456789ABCDEF
01 0S1S0*****
CPU00 SERIAL 0980524381
CPU01 SERIAL 0980524381
```

In the example above, you can see that CPU 00 and CPU 01 are shown. If there was only one CPU then only CPU 00 would be shown.

You may also use the DISPLAY M command without any sub-operands to display the channel set status:

```
d m
11.01.56          IEE070I 11.01.56 CPU/CHAN STATUS 197
01 0123456789ABCDEF
01 0S1S0*****
CPU00 SERIAL 0980524381
CPU01 SERIAL 0980524381

IEE071I 11.01.56 DEVICE STATUS
  0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
00 .  .  *  *  .  .  .  .  .  *  .  .  0  0  0  0
01 *  *  .  .  .  .  .  .  .  .  .  .  .  .  .  0
07 *  *  *  *  *  *  *  *  .  .  .  .  .  .  .  .
08 *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
09 *  *  *  *  *  *  *  *  .  .  .  .  .  .  .  .
0B *  *  *  *  *  *  *  *  .  .  .  .  .  .  .  .
0C 0  0  0  0  0  0  0  0  .  .  .  .  .  .  .  .
10 .  .  *  *  .  .  .  .  .  .  .  .  *  *  *  *
13 *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
14 *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
15 *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
16 *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
17 *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
18 *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
19 *  *  *  *  *  *  *  *  *  0  1  1  *  *  *  *
1C *  *  *  *  *  *  *  *  .  .  .  .  .  .  .  .
1E *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
1F *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
20 .  .  *  *  .  .  .  .  .  .  .  .  *  *  *  *
..<snipped for length>

IEE767I 11.01.56 STORAGE SIZE
HIGH REAL STORAGE ADDRESS IS 8192K

IEE768I 11.01.56 STORAGE STATUS
NO STORAGE OFFLINE
NO RECONFIGURABLE STORAGE UNITS DEFINED
```

In the example above, the output has been partially cut away where the double dots (..) are located to save space. The table above shows all of the i/o devices defined to MVS and what channel set they are connected to. The column of numbers on the left indicates the first two digits of the device address, and the numbered columns at the top from 0 to F indicate the last digit of the device address. So if we look at the line starting with 19 and then look to the right under the 8, 9, and A columns, it shows 0, 1, and 1, for devices 198, 199, and 19A respectively. This indicates that device 198 is on channel set 0 (CS0), and 199 and 19A are on CS1. The 3270 devices at addresses 0C0-0C7 are on CS0.

Comparison of MVS Performance with and without SPMODE

Timing results show a dramatic improvement in the performance of the MVS guest with SPMODE enabled if the DASD devices are moved to channel set 1. For comparison, several other timings are shown. These timings were performed with the VM/CE 1.2 operating system and with an MVS 3.8J TK5 update level 3 guest. The guest was running Virtual=Real with 8 MB of storage. The Hercules emulator used was SDL-Hercules 4.7 with ECPSVM set ON. The host machine is an Intel i9-13900 processor, running Windows 11.

Each timing test was run twice and the results averaged. For each configuration (SPMODE ON or OFF, and CS0 vs CS1) two jobs were run and timed separately, and then run together and timed. One job was to execute a sort. The sort job had to sort a dataset of exactly one million records; all of the iterations used exactly the same input data in the same order. The second job was an assembly of the MVS Stage 1 system generation macros, a fairly large-sized assembly. The timings shown are in whole seconds of elapsed time from the IEF403I job started message to the IEF404I job ended message. For the two jobs that ran together, the total elapsed time is based on the job that started first to the job that ended last.

#	Tested Configuration	Elapsed time seconds		
		Sort	Asm	Sort+Asm
1	VM/370+ECPS+MVS V=R, 1 CPU, SPMODE off, all DASD devices on CS0	45	144	202
2	VM/370+ECPS+MVS V=R, 2 CPUs, SPMODE on, all DASD devices on CS0	46	151	143
3	VM/370+ECPS+MVS V=R, 2 CPUs, SPMODE on, all DASD dev on CS1 ex 198	25	23	29
4	MVS Native (no VM at all), 1 CPU, all DASD devices on CS0	10	17	25
5	MVS Native (no VM at all), 2 CPUs, all DASD devices on CS1 ex 198	11	18	23

Discussion of Timing Results

Test #1 represents the current situation for VM/370 users. The MVS guest does not perform that well. It is usable, but slow.

Why did Test #2 have worse results for the individual jobs even though there were two CPUs? Because only CPU 0 can do i/o operations (as a result of assigning all of the devices to CS0) it was necessary for CP to still do all of the i/o and simulation overhead. In addition, MVS can detect that CPU 1 is idle and it dutifully will schedule work on that processor. No sooner does that work begin execution then the work needs to read or write another record. MVS must halt the work and queue that i/o operation over to CPU 0 and service the completion interruption on CPU 0 (all with CP overhead). This adds extra time. The combined jobs did benefit but only because the two jobs could run simultaneously, one on each processor. But still, any i/o operations on CPU 1 had to be requeued to CPU 0.

Test #3 represents the best case for an MVS guest while still being under the control of VM. The elapsed times are dramatically reduced. This is the result of moving nearly all of the i/o operations over to CPU 1 where MVS can execute them natively. Some i/o must be requeued to CPU 0 if it involves DASD 198 but this can't be helped. The IPL volume must remain on CPU 0. The elapsed times for Test #3 are not terribly worse than running natively under the same configuration (Test #5)

Tests #4 and #5 show the timings for MVS running natively, but in similar configurations to Test #2 and #3. The effect of two CPUs causing i/o to be queued can be shown here as well by comparing the times between tests #4 and #5.

Running VM/370 in SPMODE with two CPUs allows MVS to operate at something around 80% of its native speed, for the test cases that I used. Your mileage may vary. Documentation from back in the day states that MVS can achieve about 98% of native speed in some cases. It may be possible to optimize some DASD back to CS0 such that even with VM overhead it is still worth it if it allows better utilization of CPU 0 by MVS. In addition, there are other techniques that were available back in the day that are not available to us presently. Real hardware could use alternate channels and alternate control units, such that a channel on CPU 0 and a channel on CPU 1 could access the same string of DASD. This would avoid the queuing of i/o to another processor and boost performance considerably. Both operating systems do support this. But at present Hercules does not support the emulation of alternate channels and control units.

Limitations When Using SPMODE

Do not use the MVS “QUIESCE” command when operating in SPMODE. QUIESCE issues machine instructions to both processors to stop and enter a manual state. Since processor 1 runs MVS natively in SPMODE, if the QUIESCE command executes on that processor it will permanently stop the VM processor as well, without a proper shutdown.

Do not attempt to change the time-of-day clock using the MVS “SET” command. Only VM/370 should set the clock time at IPL.

Optional Modifications for MVS 3.8 Guests

Three optional enhancements are available that can be applied to MVS guests to make it more convenient to operate in a virtual machine. The enhancements can be used individually or together. Some of these enhancements already exist in TK4- and TK5 systems, but they will not operate properly in SPMODE. Reinstall these enhancements in order to allow them to work with SPMODE enabled or disabled. The three enhancements are in the form of SMP4 usermods. The usermods are:

1. ZBP0003 – CP SPOOLING AND CLOSE FOR JES2. This enhancement to JES2 provides the capability to automatically have JES2 issue a CP CLOSE command at the completion of writing job output to a printer or punch. In addition, the enhancement provides the ability to automatically spool job output to a CMS user. If desired, the userid is specified in the programmer-name field of the JOB statement to identify the destination for the output. The resulting output will appear in the CMS user's virtual reader queue.
2. ZBP0004 - UNDIAL. This enhancement to the VTAM logon interpret table routine provides the ability to DROP a VM DIALED connection to MVS. That is, a CP-owned 3270 terminal can DIAL to MVS and use VTAM and TSO, and when desired the terminal user can drop the dialed connection back to VM, so that the terminal can then logon to a VM userid.
3. ZBP0005 - CPCMD. This enhancement is simply a small utility program that allows CP commands to be issued from the MVS operator console. Because MVS is often run as a disconnected virtual machine, CPCMD might be used to issue a certain CP command after MVS IPL is complete, or any time a command needs to be issued. This avoids the need to logon to the MVS virtual machine console and enter the command manually.

These enhancements were written for use on MVS 3.8J TK4- and TK5. However, they can be adapted for TK3 if needed.

The installation of these enhancements assumes that the reader has a good working knowledge of how to operate a MVS guest under VM.

As always, please make sure you have a good backup of your MVS DASD before applying any of these enhancements.

Loading the Enhancements from Tape

The three usermods will each be loaded into a small PDS. The JCL used to load these datasets is shown on the next page. A copy of the JCL is also provided in the distribution zip file, named "Load_usermods_tape_jcl.txt".

```

//HERC01C JOB CLASS=A,MSGCLASS=X
//*
//* LOAD THE MVS/VM USERMODS TAPE
//*
//C1 EXEC PGM=IEBCOPY
//SYSPRINT DD SYSOUT=*
//SYSIN DD DUMMY
//SYSUT3 DD UNIT=VIO,SPACE=(CYL,5)
//SYSUT4 DD UNIT=VIO,SPACE=(CYL,5)
//SYSUT1 DD UNIT=480,LABEL=(1,SL),DISP=OLD,
// DSN=U.ZBP0003,
// VOL=(,RETAIN,SER=MVSVMU)
//*
//SYSUT2 DD DSN=USERMOD.ZBP0003,DISP=(NEW,CATLG),UNIT=SYSDA,
// SPACE=(3120,(15,2,2),RLSE),
// DCB=(BLKSIZE=3120,LRECL=80,RECFM=FB),
// VOL=SER=TK5001
//*
//C2 EXEC PGM=IEBCOPY
//SYSPRINT DD SYSOUT=*
//SYSIN DD DUMMY
//SYSUT3 DD UNIT=VIO,SPACE=(CYL,5)
//SYSUT4 DD UNIT=VIO,SPACE=(CYL,5)
//SYSUT1 DD UNIT=480,LABEL=(2,SL),DISP=OLD,
// DSN=U.ZBP0004,
// VOL=(,RETAIN,SER=MVSVMU)
//*
//SYSUT2 DD DSN=USERMOD.ZBP0004,DISP=(NEW,CATLG),UNIT=SYSDA,
// SPACE=(3120,(30,2,2),RLSE),
// DCB=(BLKSIZE=3120,LRECL=80,RECFM=FB),
// VOL=SER=TK5001
//*
//C3 EXEC PGM=IEBCOPY
//SYSPRINT DD SYSOUT=*
//SYSIN DD DUMMY
//SYSUT3 DD UNIT=VIO,SPACE=(CYL,5)
//SYSUT4 DD UNIT=VIO,SPACE=(CYL,5)
//SYSUT1 DD UNIT=480,LABEL=(3,SL),DISP=OLD,
// DSN=U.ZBP0005,
// VOL=(,RETAIN,SER=MVSVMU)
//*
//SYSUT2 DD DSN=USERMOD.ZBP0005,DISP=(NEW,CATLG),UNIT=SYSDA,
// SPACE=(3120,(15,2,2),RLSE),
// DCB=(BLKSIZE=3120,LRECL=80,RECFM=FB),
// VOL=SER=TK5001
//

```

This JCL can be used to load the MVS/VM usermods tape.

The job will create three datasets named USERMOD.ZBP0003, USERMOD.ZBP0004, and USERMOD.ZBP0005. Each of these datasets contain a member named \$INSTALL, which contains the JCL used to apply each usermod to the system.

Usermods ZBP0003 and ZBP0004 require that MVS be re-IPLed before they can be used. If you wish you may install all of the usermods and then do a single IPL with the CLPA option (that is, responding to the first MVS prompt at IPL for message IEA101A, with "R 00,CLPA").

Installing the Enhancements

1. Installing ZBP0003.

To install this enhancement, verify the JCL in USERMOD.ZBP0003(\$INSTALL) and submit the job. The changes are applied to JES2 and JES2 is relinked. The expected return codes for each step are 0. If the job runs successfully, then proceed next to update the JES2 parameters.

If you have applied the older version of this enhancement in the past, known as ZBP0001, you must install ZBP0003 in order to allow the enhancement to operate in SPMODE. ZBP0003 will operate correctly with SPMODE enabled or disabled. The older ZBP0001 cannot operate with SPMODE enabled. Most TK5 users should have ZBP0001 preinstalled already; ZBP0003 will supersede the older ZBP0001.

To update the JES2 parameters, you need to specify which printers or punches known to JES2 that will operate with the enhancement. In TK4- and TK5, the JES2 parameters are located in SYS1.JES2PARM(JES2PARM). It is recommended that you first make a copy of JES2PARM in the same dataset, (e.g., JES2BACK or JES2COPY), in case of troubles. Edit the JES2PARM member and locate the PRINTERnnn statements. Add the "VM" parameter to any PRINTER that you wish to use for spooling printouts to CMS users and to issue CP CLOSE against. For example:

```
PRINTER1  CLASS=A,NOSEP,OPERATOR,DSPLTCEL,NOPAUSE,UNIT=00E,DRAIN,  +
           UCS=QN,FCB=6,VM
PRINTER2  CLASS=Z,SEP,OPERATOR,DSPLTCEL,NOPAUSE,UNIT=00F,DRAIN,    +
           UCS=QN,FCB=6
PRINTER3  CLASS=X,SEP,AUTO,DSPLTCEL,NOPAUSE,UNIT=002,DRAIN,        +
           UCS=QN,FCB=6
```

If you wish to have a PUNCH have similar controls, add the "VM" parameter to your desired PUNCH definitions, as shown:

```
PUNCH1     CLASS=B,NOSEP,AUTO,PAUSE,UNIT=00D,START,VM
```

Note that for JES2 printers or punches to be used for VM spooling, NOSEP is a desirable option on the JES2 PRINT or PUNCH statements.

Save your changes. You must restart JES2 (warm-start is sufficient) for this change to take effect. It is probably easier to just re-IPL MVS. Alternately, wait to re-IPL until all three usermods are installed.

Once JES2 is active, you can try to submit a job from a CMS userid and have the job output returned back to that CMS userid in his virtual reader queue. Create a job on your CMS A-DISK called TEST JCL. For example:

```
//HERC01A JOB 1,BOB,CLASS=A
//X EXEC PGM=IEFBR14
//
```

In this example, the CMS userid is BOB. Note that BOB is placed as the name in the programmer-name field of the JOB statement. JES2 will use up to the first 8 characters of the programmer-name field as the userid for VM designated printers. The userid is delimited by a blank, period, or comma, if you wish to have other information in the name field as well.

Submit the job by issuing these commands:

```
CP SPOOL PUN MVS
PUNCH TEST JCL (NOH
```

If the JES2 reader is active, the job should be received and begin execution. Once the job completes, which should be nearly immediate, the output will be queued to the printer. You may have to start the printer in JES2, (e.g, \$\$ PRT1) the first time. The output from the job should then be spooled to the CMS userid named BOB.

From the CMS side, you should have a file in your virtual reader queue:

```
PRT FILE 0033 FROM MVS COPY 01 NOHOLD
```

Then you could query your reader queue to see what has arrived:

```
CP Q R ALL
OWNERID FILE CLASS RECDs CPY HOLD DATE TIME NAME DIST
BOB 0033 A PRT 000039 01 NONE 01/04 15:50:53 HERC01A MVS
Ready; T=0.01/0.01 15:51:48
```

You can then read in the spool file onto the A-disk with the READCARD command, and then view or purge the file, or use other facilities available to VM users.

Notes about using the JES2/VM spooling enhancement

- If you are running MVS native (not under VM), then no CP SPOOL or CP CLOSE commands will be issued to the printer or punch. Output will be spooled to the native device as per usual JES2 operation.
- For clarity, you may optionally specify NOVM on the PRINTER or PUNCH statements that will not be using CP SPOOL/CLOSE services.
- If the userid cannot be recognized in the JOB statement programmer name field, or if the userid is not defined in the CP system directory, then the output is automatically spooled to SYSTEM and the output will either be held in the VM spool with the MVS virtual machine's userid as the owner, or immediately printed on a CP controlled printer if it is active.

- The sysout class of the output in JES2 is used as the output class in VM.
- Jobs do not have to be submitted only from a CMS userid in order to qualify for JES2 spooling back to VM. For example, a job can be submitted from TSO, and as long as the userid in the programmer-name field is valid in the JCL and the SYSOUT class of the job directs it to a JES2 printer with the VM option in the printer definition, the output will be spooled to that CMS userid.
- If some JES2 printers or punches use *real devices*, that is, they are dedicated to MVS, attached to MVS, or are defined on channel set 1 (when running SPMODE), then these printers *should not* specify the VM option on the JES2 printer definition. This will ensure that CP spooling or close functions will not attempt to operate on that real device.

2. Installing ZBP0004.

To install this enhancement, verify the JCL in USERMOD.ZBP0004(\$INSTALL) and submit the job. The updated logon interpret exits will be assembled and linked into SYS1.LPALIB. The expected return codes for each step are 0.

If you have applied the older version of this enhancement in the past, known as ZBP0002, you must install ZBP0004 in order to allow the enhancement to operate in SPMODE. ZBP0004 will operate correctly with SPMODE enabled or disabled. The older ZBP0002 cannot operate with SPMODE enabled. Most TK5 users should have ZBP0002 preinstalled already; ZBP0004 will supersede the older ZBP0002.

It will be necessary to re-IPL with CLPA in order for the exit to take effect but before you do, it may be helpful to update the VTAM 3270 major node parameters so that they will use the new exit. The major node parameters are in SYS1.VTAMLST(L3274). You may also proceed without IPL until you install all of the other usermods.

Replace the existing name of the logon interpret exit (ETHLOGON) with the name of the new exit, ETHLOGVM. A partial example is shown below of the L3274 member:

```
L3274      LBUILD SUBAREA=2
CUU0C0     LOCAL  TERM=3277, CUADDR=0C0, ISTATUS=ACTIVE,
              LOGTAB=ETHLOGVM, LOGAPPL=NETSOL,
              FEATUR2=(MODEL2, PFK)
CUU0C1     LOCAL  TERM=3277, CUADDR=0C1, ISTATUS=ACTIVE,
              LOGTAB=ETHLOGVM, LOGAPPL=NETSOL,
              FEATUR2=(MODEL2, PFK)
.
.
.
```

After you have edited all of the individual terminal descriptions that you wish to have use the new exit, save your changes. Then delete the previous working member named L3274 from the SYS1.VTAMOBJ dataset, (the member may or may not be present), in order to cause VTAM to rescan the new L3274 parameters on its next start up.

To create virtual terminals that you can dial to, from the MVS machine virtual console define several GRAF devices, e.g.:

```
#CP DEF GRAF 0C0
#CP DEF GRAF 0C1
```

Optimally, you can make this permanent and more convenient by using SPECIAL statements in the CP directory entry for user MVS (as shown in the sample MVS directory definitions in the prior section).

When you are ready to re-IPL MVS, ensure that you reply to the first prompt, R 00,CLPA. Once initialized, start TSO. If you forget to use CLPA, the changes will not take effect, and you must re-IPL with CLPA.

Once the system is active, you can use a VM terminal to DIAL MVS if MVS is running under VM as a guest. This will connect you to one of the GRAF devices defined above. If VTAM is active, you should immediately be presented with the VTAM TK4 or TK5 logo screen. Now you can log on to TSO as usual or use another VTAM application. In order to return to VM, exit your application back to the VTAM TK4/TK5 logo screen (such as TSO LOGOFF) and simply type in 'undial'. The terminal will drop back to VM control. At that point you can log on to a CMS userid or DIAL MVS again.

If you are running MVS natively (not under VM), then typing 'undial' from the VTAM TK4/TK5 logo screen has no effect, except that VTAM will issue a message about invalid input from 'undial'. Effectively though, it is a no-op.

3. Installing ZBP0005.

To install this enhancement, verify the JCL in USERMOD.ZBP0005(\$INSTALL) and submit the job. The programming will be assembled and linked into SYS1.LPALIB. The expected return codes for each step are 0 except for the APPLY step which should be 0 or 4. This JCL will install the CPCMD started task procedure to SYS1.PROCLIB and add a small program called VMPCMD to SYS1.LINKLIB. There is no re-IPL requirement in order to use CPCMD.

Using CPCMD.

In order to use CPCMD, simply issue an MVS operator START command using the CPCMD procedure, and specify the actual CP command image in the START command. For example, if you wish to attach tape drive 480 to MVS, you could issue the command like this:

```
S CPCMD,CMD='ATT 480 MVS'
```

The apostrophes are required to delimit the command. You may also specify multiple commands in a single invocation of CPCMD. If you choose to do this, you separate the commands with a pound-sign (#) character:

```
S CPCMD,CMD='SET TIMER REAL#PURGE * PRT ALL'
```

CPCMD can issue any command that the MVS userid is allowed to issue; including re-IPLing MVS. If a command generates a response, that response will appear on the MVS userid virtual machine console, if connected to a terminal.

CPCMD commands can be placed in the COMMNDxx members of SYS1.PARMLIB or any other automated tools that can issue START commands, if desired.

The maximum length of the total command image inside the apostrophes is limited to 132 characters.

VM/370 SPMODE Version History

xx Jun 2025 – v1.0.0

- First public release

March-May 2025 – V0.9

- Beta versions to selected users.

Other Modifications and Software

Modifications and Software for VM/370 by the author:

- Full Screen Support for Virtual Machines (DIAGNOSE 58)
- Logical Device Facility (DIAGNOSE 7C)
- Bug fixes to existing Attached Processor support
- Shadow Table Bypass Support
- 3270 Device Characteristics (DIAGNOSE 8C)
- 3270 Model 5 support for CP and CMS
- Alternate CP Nucleus Support
- SHUTDOWN REIPL command
- Allow 4-digit cylinder numbers in the CP directory
- XNET (Cross System Net), a Passthru-VM like networking software

Modifications and Software for MVS 3.8 by the author:

- NJE38 – Network Job Entry software for MVS 3.8
- Usermod ZBP0001 – allow JES2 in a MVS guest to spin-off job output to CMS users

VM/370-related modifications to Hercules by the author:

- ECPS:VM bug fixes to the existing ECPS implementation
- New assists added per the ECPS:VM specification

All of the above VM modifications are preinstalled on VM Sixpack 1.3 and VM/CE distributions. The MVS 3.8 modifications and software are preinstalled on the TK5 distribution. The Hercules ECPS:VM enhancements are available in SDL-Hercules release 4.4 and above, and in all Aethra releases. The ECPS:VM bug fixes (only) are also available in Hercules 3.13.